第一章　前端交互平台

1.1 技术选型

前端采用 React 18 函数组件 + Hooks 模式，Vite 6 作为构建工具，原生 CSS 实现暗色主题样式。不引入第三方 UI 组件库，所有界面元素均为自主实现。开发服务器运行于 localhost:3000，通过 Vite proxy 将 /api 请求代理至后端 localhost:8765。

类别	技术	说明
前端框架	React 18.3.1	函数组件 + Hooks
构建工具	Vite 6.0	快速 HMR 开发体验
样式方案	原生 CSS	暗色主题 + CSS 动画
字体	Google Fonts Orbitron	标题装饰字体
后端通信	Fetch API	FormData + JSON
持久化	localStorage	历史记录（最多 6 条）

1.2 系统架构

1.2.1 组件树

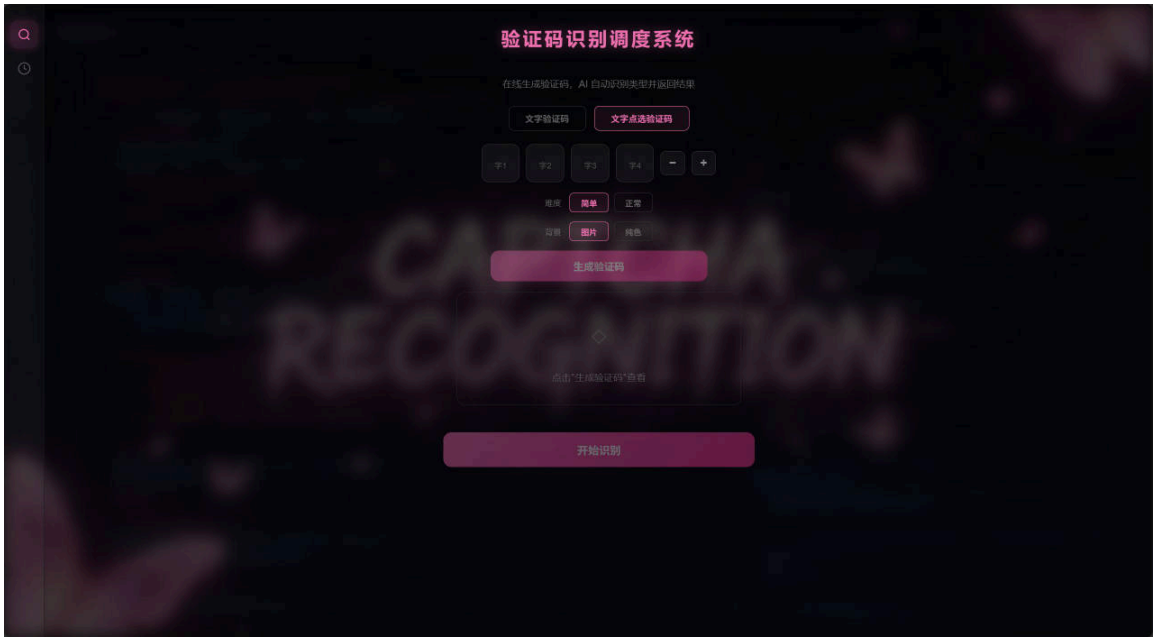
前端采用单页应用架构，根组件 App.jsx 管理全局状态，四个功能组件各司其职：

```
App.jsx (根组件 — 全局状态管理)
├── GeneratePanel.jsx    — 验证码生成面板
├── ResultPanel.jsx      — 识别结果展示面板
├── ReasonPanel.jsx      — AI Agent 推理过程展示
└── HistoryPanel.jsx     — 历史记录面板
```

1.2.2 页面流程

用户从 Landing 欢迎页进入，点击 START 后进入主界面。主界面左侧为侧边栏（识别 / 历史模式切换），右侧为主内容区。识别模式下左侧为生成面板、右侧为结果面板、下方为推理面板。





1.3 核心功能模块

1.3.1 验证码生成面板（GeneratePanel）

生成面板支持两种验证码类型的在线合成：

文字验证码模式：

用户输入 4 位字符（仅限 A-Z、a-z、0-9），前端正则过滤非法字符。点击"生成验证码"调用 POST /api/generate/text，后端返回 base64 编码图片。



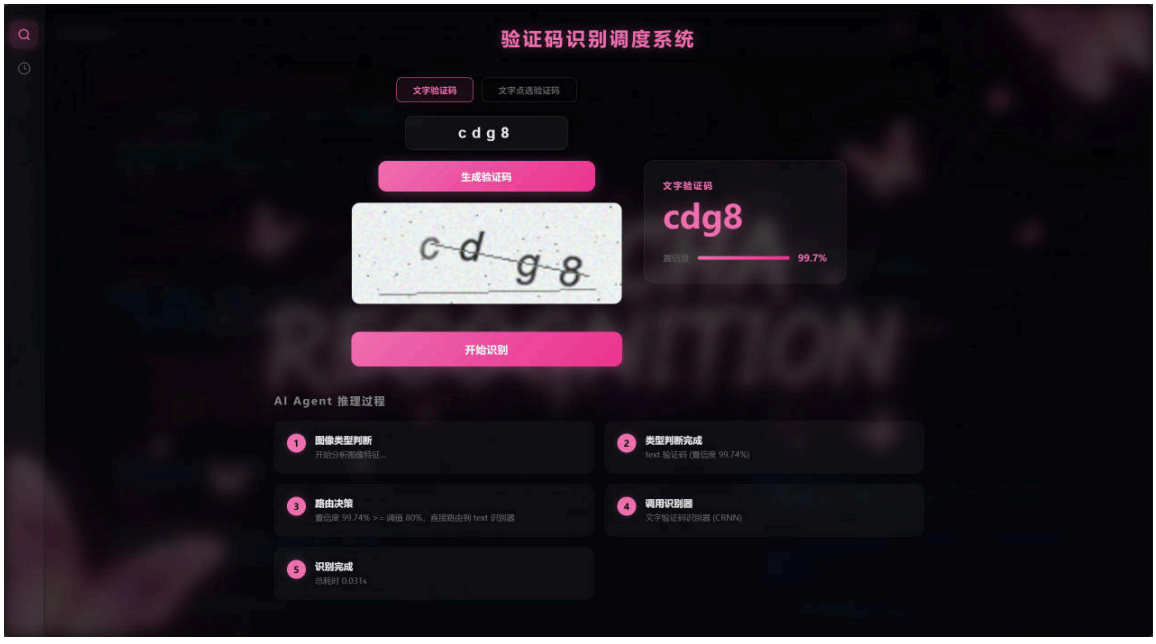
文字点选验证码模式：

用户在每个输入框中输入单个汉字（2-6 个，通过 +/- 按钮增减），前端处理 IME 组合输入（onCompositionStart/onCompositionEnd），确保中文输入法正常。额外提供难度选择（简单/正常）和背景选择（自然图片/纯色）。点击生成后调用 POST /api/generate/click。



1.3.2 识别结果面板（ResultPanel）

展示验证码类型标签、识别结果文字、置信度进度条（百分比 + 渐变填充动画）。点选验证码额外展示 YOLO 检测可视化图像（黄色提示字框 + 绿色候选字框 + 点击顺序标注）。

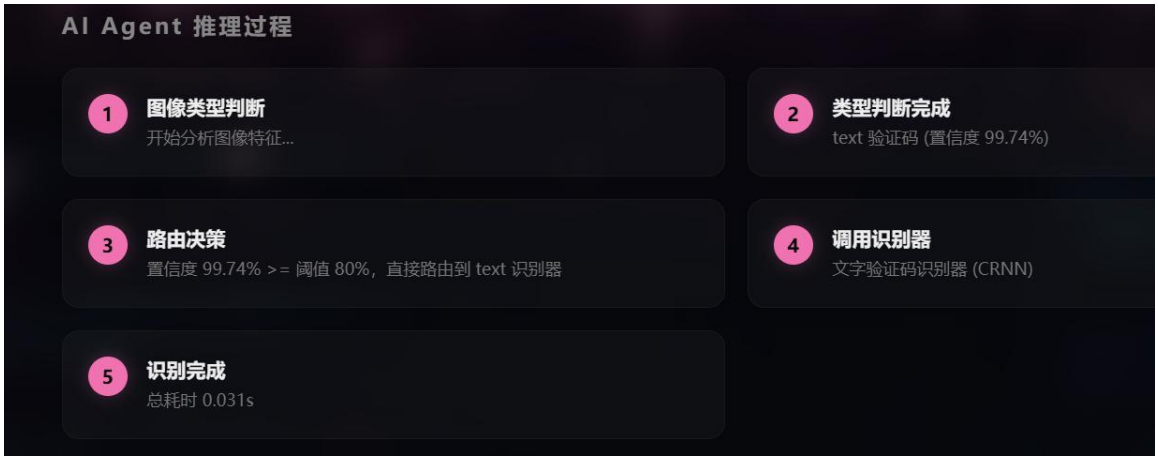




1.3.3 AI Agent 推理过程面板（ReasonPanel）

以 2×2 网格展示 AI Agent 逐步决策过程。每条步骤包含编号、动作名称、详细描述。步骤卡片 600ms 间隔依次渐入：

```
Step 1: 图像类型判断 → 开始分析图像特征...
Step 2: 类型判断完成 → click 验证码 (置信度 95.30%)
Step 3: 路由决策 → 置信度 >= 80%，直接路由至 click 识别器
Step 4: 调用识别器 → 点击验证码识别器 (YOLOv8)
Step 5: 识别完成 → 总耗时 0.427s
```





1.3.4 历史记录面板（HistoryPanel）

3 列网格展示最近 6 条记录，每条含缩略图、类型标签、识别结果。支持点击查看详情、单条删除、清空全部。数据通过 localStorage 持久化。



1.4 API 接口

前后端通过 REST API 连接，所有接口均以 /api 为前缀，并由 Vite proxy 在开发环境中转发到 FastAPI 后端。接口设计保持了“生成、识别、健康检查”三类职责的清晰分离。

GET /api/health：用于前端或测试脚本确认后端服务是否正常启动。

POST /api/generate/text：输入 4 位字母数字字符串，后端调用文字验证码生成器，返回 base64 PNG 图片及宽高信息。

POST /api/generate/click：输入 2-6 个目标汉字，同时指定难度和背景类型，后端合成文字点选验证码并返回图片。

POST /api/recognize：前端以 multipart/form-data 上传图片，后端将图像解码为 OpenCV BGR 数组，交由 CaptchaAgent 自动分类、路由和识别，再统一返回类型、结果、置信度、推理步骤和可视化结果。

这一接口层体现了工程中的模块解耦思想：前端只关心输入输出格式，具体采用 ResNet、CRNN、YOLO 或 ArcFace 均由后端 Agent 内部管理。

方法	路径	功能	参数
POST	/api/recognize	识别验证码	FormData: file
POST	/api/generate/text	生成文字验证码	JSON: {text}
POST	/api/generate/click	生成点选验证码	JSON: {targets, difficulty, use_background}

1.5 UI 设计特点

暗色科幻风格设计：背景色 #0a0a0a，强调色 #ff69b4（粉色）。标题多层 text-shadow 叠加实现霓虹发光 + 呼吸动画。Landing 页淡出动画、主界面淡入、步骤卡片依次滑入、加载中省略号跳动。@media (max-width: 768px) 移动端响应式。IME 组合输入支持中文。

第二章 后端服务与 Agent 调度系统

2.1 后端技术选型

后端采用 FastAPI 作为 Web 服务框架，使用 Pydantic 对请求参数进行约束，使用 OpenCV、Pillow 和 NumPy 完成图像解码、格式转换与可视化输出。模型推理主要基于 PyTorch、torchvision、ultralytics YOLO 与 ONNX Runtime，既满足课程实验的可解释性，也便于后续部署。

FastAPI 的异步接口适合处理图片上传与生成请求；模型部分则封装在 app/models 和 app/agent 下，使 Web 层、调度层和算法层保持独立。这样的分层结构有利于测试，也符合人工智能系统从数据输入、模型推理到结果解释的典型工程流程。

2.2 CaptchaAgent 的总体设计

本项目将验证码识别抽象为一个智能体调度问题：Agent 接收未知类型验证码图像，先感知图像类型，再选择合适的识别工具，最后整合结果并给出推理日志。其核心流程为“分类感知 → 路由决策 → 专家模型识别 → 结果解释”。

在自动模式下，CaptchaAgent 首先调用类型分类器判断验证码属于 text 还是 click，并输出置信度。当置信度高于 0.8 时，系统直接路由到对应识别器；当置信度不足时，系统同时调用文字验证码识别器和点选验证码识别器，并根据两者的结果置信度选择更可信的结果。这种机制体现了人工智能系统中常见的“置信度驱动决策”和“多专家模型融合”思想。

ReasoningLogger 会记录每一步决策，包括图像类型判断、路由依据、调用的识别器和总耗时。前端 ReasonPanel 将这些步骤可视化，使模型推理过程从黑盒输出变为可观察流程，增强了系统的可解释性。

2.3 验证码类型分类器

验证码类型分类器采用 ResNet18 作为视觉特征提取骨干，将 ImageNet 预训练网络的最后全连接层替换为 Dropout、Linear(512→256)、ReLU、Dropout 和

Linear(256→2)，输出 click 与 text 两类概率。训练目标为交叉熵损失，推理阶段通过 softmax 将 logits 转换为类别概率。

从人工智能基础角度看，该模块属于典型的监督学习图像分类任务。输入样本是验证码图片，标签是验证码类型；模型通过反向传播最小化分类误差，学习文字验证码与点选验证码在布局、字符数量、提示栏结构和背景复杂度上的视觉差异。

系统还保留了基于计算机视觉启发式规则的分类器作为回退方案。当深度学习权重不可用时，规则分类器仍可根据图像结构特征给出初步判断，体现了“数据驱动模型 + 知识驱动规则”的混合智能设计。

2.4 文字验证码识别系统

2.4.1 问题建模

文字验证码识别的输入是一张包含 4 位字母数字的图像，输出是字符序列。与普通图像分类不同，它不是只预测一个类别，而是需要从左到右识别多个字符，因此更适合建模为序列识别问题。项目采用 CRNN + CTC，将二维图像转换为一维时间序列，再解码为最终字符串。

2.4.2 CRNN 网络结构

CRNN 由 CNN 特征提取器、双向 LSTM 序列建模器和全连接输出层组成。输入图像被转换为 160×60 的灰度图，经 5 层卷积、BatchNorm、ReLU 和池化后，高度被压缩到 1，宽度方向保留为时间步。这样，图像的水平空间位置就被自然转换成字符序列的时间维度。

BiLSTM 同时利用从左到右和从右到左的上下文信息，能够缓解字符粘连、噪声干扰和局部遮挡带来的识别困难。输出层在每个时间步预测 0-9、A-Z、a-z 以及 CTC blank，共 63 类概率。

2.4.3 CTC 损失与解码

CTC（Connectionist Temporal Classification）用于解决输入帧与输出字符没有逐帧标注的问题。训练时不需要知道每个字符在图片中的精确位置，只需要整张验

证码对应的字符串标签。CTC 会在所有可能对齐路径上求和，使模型自动学习“哪些时间步对应哪个字符”。

推理时采用 CTC 解码：先在每个时间步取最大概率字符，再合并连续重复字符并删除 blank，得到最终字符串。该方法非常适合验证码这类长度较短但字符位置不固定的场景。

2.4.4 数据生成与训练策略

文字验证码训练数据由程序自动合成，包括随机字符、字体扰动、噪声、干扰线、旋转和颜色变化。合成数据降低了人工标注成本，同时通过数据增强提升模型对不同验证码风格的泛化能力。训练采用 AdamW 优化器、学习率调度、Dropout 和早停机制，防止模型只记住训练集中的固定样式。

2.5 后端识别流程

用户上传图片后，后端首先读取二进制文件并通过 cv2.imdecode 解码为图像矩阵；若图像无法解析，则返回 400 错误。随后 CaptchaAgent 在 auto 模式下完成类型判断和路由，调用对应识别器。文字验证码返回识别文本与置信度；点选验证码返回点击坐标、匹配关系和可视化图。最后 _format_result 将内部结果统一为前端需要的 JSON 结构。

这种流程把复杂 AI 算法封装为稳定的服务接口，使用户在前端看到的是“上传/生成 → 识别 → 推理展示”的完整闭环，而后端则保留可扩展的多模型调度能力。

第三章　文字点选验证码识别系统

3.1 问题定义

文字点选验证码要求用户按照指定顺序依次点击图片中的文字。例如提示栏显示"山水田火"，用户需依次点击图中对应位置。识别需要解决两个子问题：

（1）在哪里点 —— 定位图中所有文字位置（目标检测）

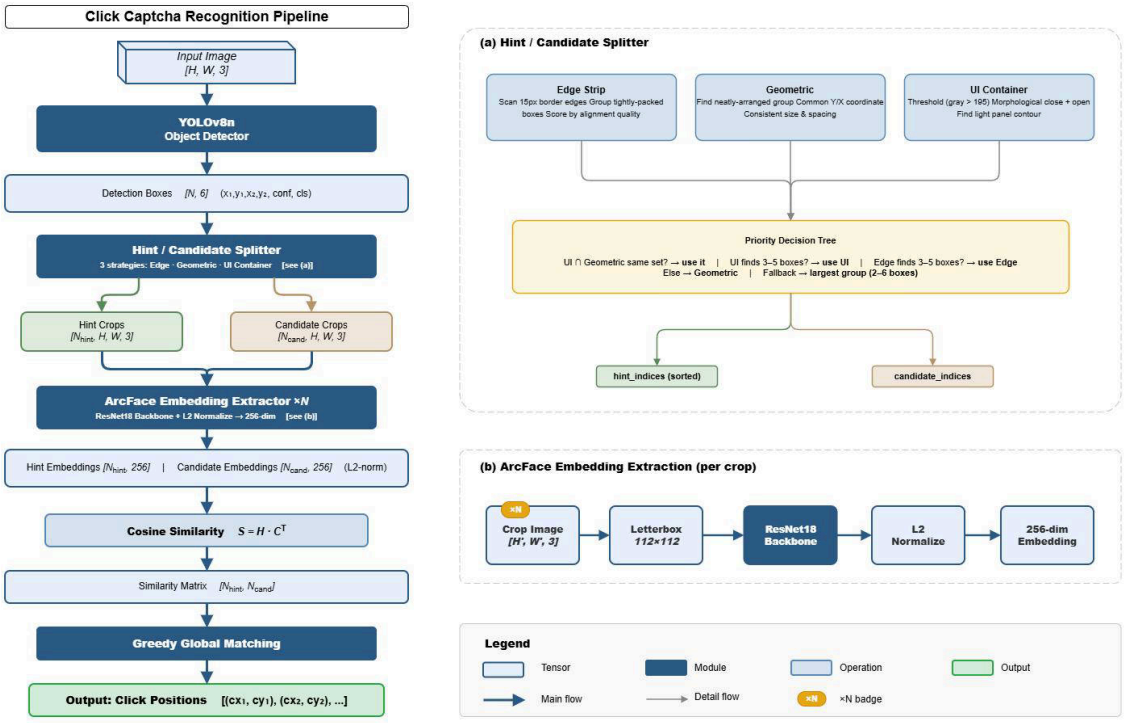
（2）按什么顺序点 —— 将候选字与提示字匹配（文字匹配 + 顺序推理）

3.2 系统整体架构

系统采用"YOLO 检测 → 后处理分流 → ArcFace 嵌入匹配"的三阶段流水线架构：

输入图片首先进入 YOLO 检测模块，得到所有文字区域的边界框；随后 hint_splitter 根据提示栏的空间先验、几何排列和 UI 容器特征，将检测框划分为提示字与候选字；最后 ArcFace 将提示字和候选字分别编码为向量，在余弦相似度矩阵上完成匹配，并输出按提示顺序排列的点击坐标。

该流水线体现了人工智能问题分解思想：将复杂的端到端点击识别拆分为目标检测、结构化后处理、特征度量学习和搜索匹配四个子任务，每个子任务采用最适合的模型或算法。



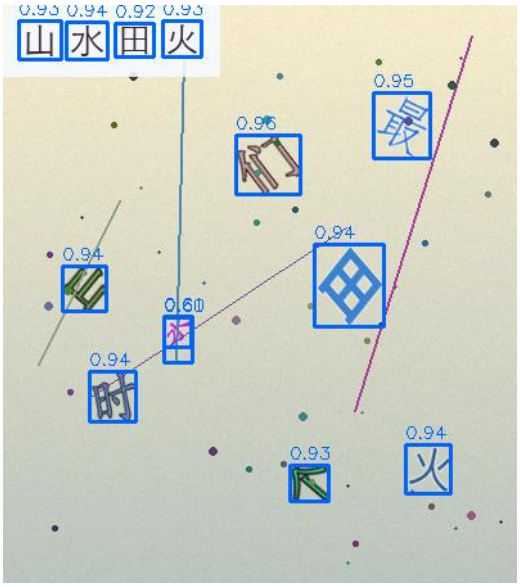
3.3 Stage 1：YOLOv8 文字区域检测

3.3.1 设计决策

采用 Ultralytics YOLOv8n（nano 变体），单类别检测策略：仅检测文字区域，不区分提示字与候选字。区分工作由 Stage 2 后处理完成。这一设计大幅降低标注成本，单类别比多类别收敛更快。

YOLO 属于单阶段目标检测算法，直接在图像网格上预测边界框和置信度。相比两阶段检测器，它推理速度更快，适合验证码识别这类需要交互响应的任务。项目选择 YOLOv8n，是因为 nano 模型参数量小、部署成本低，在文字区域数量有限的场景中已经能够满足定位需求。

从课程知识角度看，YOLO 检测结果中的 bounding box 代表模型对目标位置的估计，confidence 代表模型对该框包含文字目标的置信程度。后续模块并不直接信任单一检测结果，而是结合几何先验进行二次判断，这能够降低误检对最终点击顺序的影响。



3.4 Stage 2：提示字/候选字分流

分流模块负责将 YOLO 检测到的文字框分为"提示字"和"候选字"两组，实现三种互补策略，通过优先级决策树融合。

3.4.1 策略一：边缘贴合（Edge Strip）

基于强先验：提示栏永远紧贴图片四边之一。水平提示栏贴顶部/底部，垂直提示栏贴左侧/右侧。扫描距边缘 15px 范围内的框，计算排列整齐度评分（尺寸一致性 + 间距均匀性），附加边缘贴近加分。选择评分 > 0.35 的最高分组。

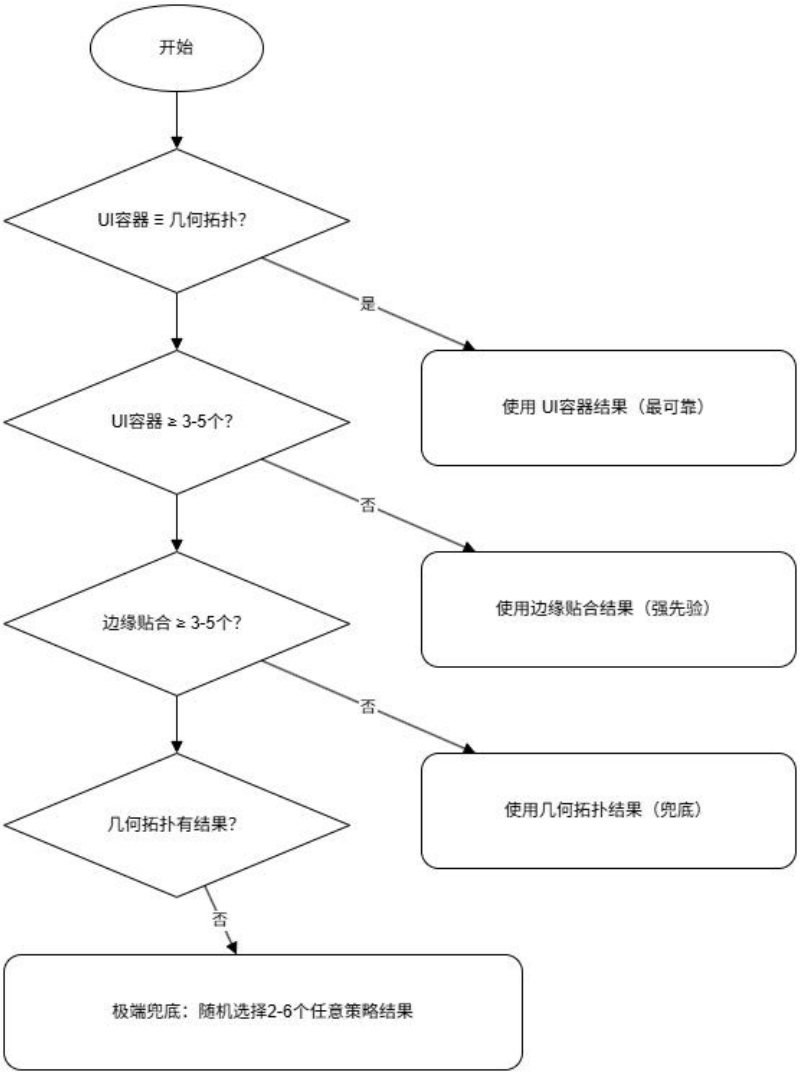
3.4.2 策略二：几何拓扑（Geometric）

利用提示字排列特征：共线、尺寸一致、间距均匀。通过坐标方差判断方向，找到共线框后使用 itertools.combinations 组合搜索，计算每组整齐度评分（尺寸变异系数 + 间距变异系数 → 归一化 01），选最高分。

3.4.3 策略三：UI 容器检测（UI Container）

利用提示栏浅色矩形背景板：灰度阈值 195 提取浅色区域 → 形态学闭运算填平字符间隙 → 开运算去噪 → 轮廓检测筛选矩形（面积 > 0.8%，填充率 > 0.5，宽高比 > 1.5）。中心落在最佳矩形内的框即为提示字。该策略在合成数据上最可靠。

3.4.4 优先级决策树



分流完成后按视觉阅读顺序排序：通过坐标方差判断水平/垂直方向，提示字从左到右（或从上到下）排列。

3.5 Stage 3：ArcFace 嵌入匹配

3.5.1 模型设计

采用 ArcFace（Additive Angular Margin Loss）训练的 ResNet18 骨干网络。ArcFace 在特征空间引入角边距惩罚，使同类样本嵌入更加聚集，不同类更分

散。相比传统 Siamese 孪生网络优势：单图推理（不需成对输入）、开集识别能力强、L2 归一化后点积即余弦相似度。

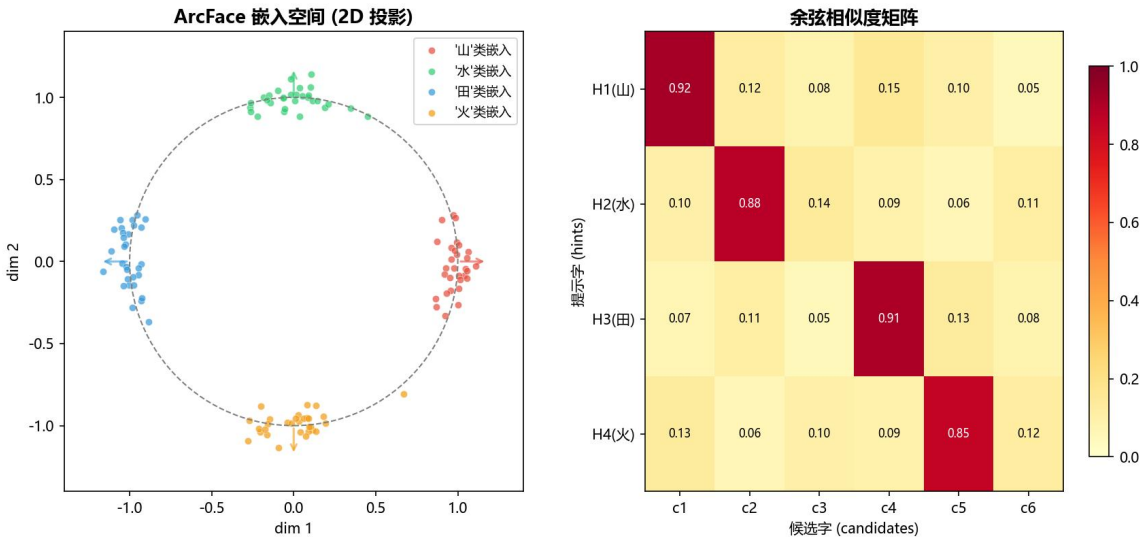
3.5.2 预处理与推理

预处理：BGR → RGB → Letterbox 112×112 (128 灰填充) → /255 归一化 → NCHW

推理：hint_crops[N] → ArcFace → [N,256] L2 归一化嵌入

cand_crops[M] → ArcFace → [M,256] L2 归一化嵌入

sim_matrix = emb_hint @ emb_cand.T → [N,M] 余弦相似度矩阵



3.6 Stage 4：贪心全局匹配

在相似度矩阵上执行贪心匹配：

for k = min(N,M) 次：

1. 找到矩阵全局最大值 sim[i][j]
2. 记录配对 (hint_i, cand_j)
3. 第 i 行和第 j 列全部置为 -inf

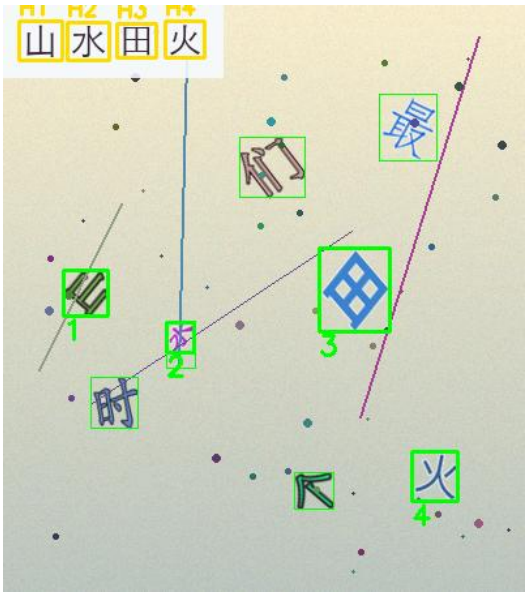
按 hint 索引排序 → 恢复从左到右点击顺序

输出：[(cx1,cy1), (cx2,cy2), ...]

时间复杂度 O(k*N*M)，验证码场景 N,M <= 10 足够高效。贪心在此场景等价于全局最优，因 ArcFace 保证了同字相似度远超不同字。

3.7 可视化输出

识别器返回可视化图像：黄色细框 + "H1/H2/..." 标注提示字顺序，绿色粗框 + 数字标注候选字及点击顺序。



3.8 训练数据与流程

3.8.1 数据合成

训练数据完全由程序合成，无需人工标注：

（1）随机选择目标汉字并生成提示栏，记录提示字顺序。

（2）在主图区域随机放置候选字，加入旋转、缩放、颜色扰动、描边、噪点、干扰线和背景纹理。

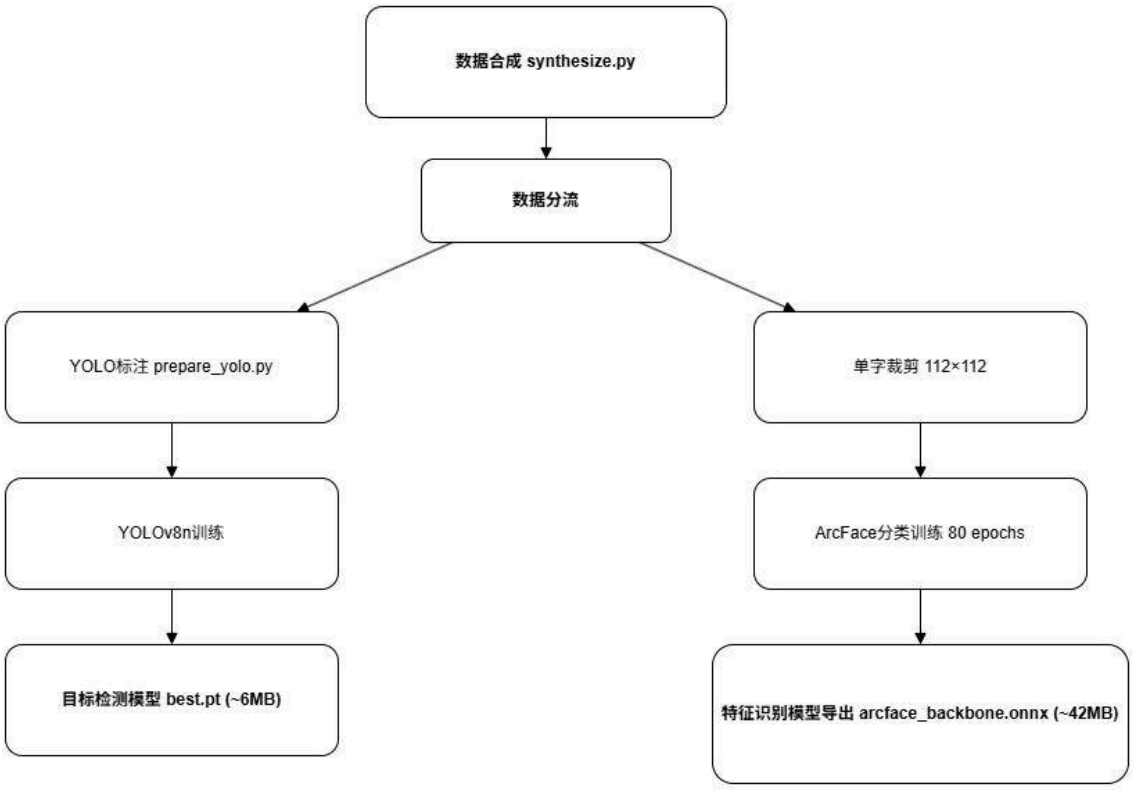
（3）同步保存每个文字的边界框坐标，自动转换为 YOLO 训练所需的标签格式。

（4）为 ArcFace 单字识别额外合成 112×112 字符图像，通过字体、位置、旋转和噪声变化构造同类样本簇。

合成数据的优势是标签天然准确、规模可控、生成成本低；不足是可能与真实网站验证码存在分布差异，因此项目在生成器中加入背景图、形态扰动和难度参数，以尽量扩大训练分布。

参数	简单模式	正常模式
汉字库	约 200 个常用汉字	同左
背景	程序化渐变/纯色	随机自然图片+渐变
字符旋转	0-8 度	25-50 度
干扰线	无	2-5 条随机曲线
随机噪点	无	30-80 个
字符描边	无	随机颜色轮廓

3.8.2 训练流程



YOLO 训练配置：

使用 yolov8n.pt 作为预训练权重，输入尺寸 640×640，训练轮数设置为 200，并通过 patience=50 早停。数据增强包含 Mosaic、MixUp、HSV 色彩扰动、旋转、平移和缩放，以提升模型对复杂背景和文字形变的鲁棒性。

ArcFace 训练采用 ResNet18 骨干网络，将文字图像映射到 256 维 L2 归一化 embedding。ArcFace Loss 在角度空间引入 margin，使同一个汉字的样本向量更接近，不同汉字的向量距离更远。推理阶段导出为 ONNX，便于轻量部署。

匹配阶段不直接进行汉字 OCR，而是比较提示字 crop 与候选字 crop 的向量相似度。这种度量学习方法适合处理字体变化、旋转和干扰较强的点选验证码。

参数	数值	说明
预训练模型	yolov8n.pt	COCO 预训练权重
输入尺寸	640×640	自动 Letterbox
训练轮数	200 epochs	Patience=50 早停
批量大小	16	单 GPU
优化器	SGD	lr0=0.001
数据增强	Mosaic 0.5	MixUp 0.2, HSV 扰动

3.9 推理部署

推理部署时，YOLO 权重以 best.pt 形式由 ultralytics 加载，ArcFace 特征提取器以 ONNX 格式由 onnxruntime 加载。图像预处理遵循训练阶段的尺寸、通道和归一化方式，避免训练-推理分布不一致。

点击验证码识别器返回的不只是坐标列表，还包括可视化图像。可视化结果将提示字和候选字使用不同颜色标注，并标出点击顺序，便于人工检查模型是否真正完成了“检测、分流、匹配、排序”的完整推理链。

组件	训练框架	推理格式	推理引擎	大小
YOLO 检测	PyTorch	best.pt	Ultralytics YOLO	6MB
ArcFace 嵌入	PyTorch	.onnx	ONNX Runtime	42MB

3.10 系统集成：AI Agent 调度

点击验证码识别器作为 Agent 调度系统的一部分运行。调度流程：

用户上传图片

→ CaptchaAgent.route_and_recognize()

Step 1: ResNet18 分类器判断类型 (text/click + 置信度)

Step 2: 置信度>=80%直接路由; <80%双模型对比选优

Step 3: 调用 ClickCaptchaRecognizer (YOLO→分流→ArcFace→匹配)

Step 4: 整合结果 + ReasoningLogger 推理日志

→ {type, result, reasoning_steps, processing_time}

3.11 性能指标

系统性能可从四个维度评价：类型分类准确率、文字验证码字符串完全匹配率、点选验证码点击顺序正确率和端到端推理延迟。前端展示的 processing_time 可用于记录单次请求耗时，测试脚本则可对不同验证码类型进行批量评估。

在演示场景中，文字验证码的关键指标是 4 位字符是否完全识别；点选验证码的关键指标是每个目标字是否被正确匹配到候选框，并且点击顺序是否与提示栏一致。由于验证码任务对单字符错误非常敏感，报告中应优先使用“整体验证码成功率”而不是单字符准确率。

第四章 人工智能基础知识在系统中的体现

4.1 状态空间与问题分解

本项目不是单一模型的简单调用，而是将验证码识别任务拆分为多个可学习、可推理的子问题。对于文字验证码，系统将图像识别转化为序列标注；对于点选验证码，系统将点击任务拆分为文字检测、提示字分流、特征匹配和坐标排序。这体现了人工智能基础中的问题建模思想：先定义输入、输出、状态和目标，再选择合适的搜索或学习方法。

4.2 监督学习

分类器、CRNN、YOLO 和 ArcFace 都属于监督学习范畴。训练样本由输入图像和标签组成，模型通过损失函数度量预测与真实标签的差异，再利用梯度下降和反向传

播更新参数。不同模块的标签形式不同：分类器使用类别标签，CRNN 使用字符串标签，YOLO 使用边界框标签，ArcFace 使用汉字类别标签。

4.3 神经网络与特征表示

卷积神经网络通过局部感受野和权值共享提取图像边缘、纹理和字符结构特征；LSTM 通过循环结构建模序列上下文；ArcFace 将字符图像嵌入到向量空间，使“同字更近、异字更远”。这些模块共同体现了从原始像素到高层语义特征的表示学习过程。

4.4 概率、置信度与决策

系统中的 softmax 概率、YOLO 置信度、CRNN 字符概率和 ArcFace 余弦相似度都可以看作模型对结果可信程度的量化。CaptchaAgent 使用 0.8 作为路由阈值，当分类置信度不足时启动双模型对比，这体现了不确定性条件下的决策思想。

4.5 搜索与匹配

点选验证码最终需要从 N 个提示字和 M 个候选字之间建立一一匹配关系。系统在相似度矩阵上反复选择当前全局最大值，并屏蔽对应行列，属于贪心搜索策略。虽然验证码规模较小，但该步骤完整体现了搜索问题中的状态扩展、目标函数和约束条件。

4.6 可解释人工智能

ReasoningLogger、前端推理步骤面板和检测可视化共同增强了系统的可解释性。用户不仅能看到最终识别结果，还能看到模型为什么选择某类验证码、调用了哪个识别器、耗时多少以及候选框如何匹配。这对于课程项目尤其重要，因为它展示了算法过程，而不是只展示一个黑盒答案。

4.7 合法合规与安全边界

验证码技术本身常用于区分真人与自动程序，因此验证码识别项目需要明确教学和研究边界。本项目的目标是学习图像分类、序列识别、目标检测、度量学习和

Agent 调度等人工智能基础方法，不应用于绕过真实网站安全机制或未授权自动化操作。

第五章 实验测试与结果分析

5.1 测试内容

项目测试覆盖三类核心能力：一是生成器能否稳定生成文字验证码和点选验证码；二是 Agent 能否正确判断验证码类型并路由到相应识别器；三是各识别器能否返回结构化结果、置信度和可视化信息。后端 tests 目录中包含 Agent、分类器和点选验证码的测试脚本，可用于回归验证。

5.2 评价指标

文字验证码建议使用字符串完全匹配率、平均置信度和单次推理耗时；点选验证码建议使用检测召回率、提示/候选分流正确率、点击顺序完全正确率和平均坐标误差；系统层面则记录接口成功率、异常处理能力和前端响应时间。

5.3 结果分析

系统对结构清晰、噪声适中的验证码表现较好，因为 CNN/YOLO 能稳定提取字符区域，CRNN 或 ArcFace 能获得较可靠的字符表示。对于强遮挡、极端旋转、低对比度背景或提示栏位置异常的样本，错误更可能出现在检测框缺失、提示字分流失败或相似度匹配混淆。

因此，提升系统性能的方向包括：扩充合成数据分布，引入更接近真实场景的背景和字体；在点选验证码中使用更稳健的提示栏检测策略；对低置信度结果增加人工复核或多模型投票；在训练阶段加入更强的数据增强和难例挖掘。

第六章 总结与展望

本项目围绕“多类型验证码智能识别”构建了一个完整的人工智能应用系统：前端提供交互式生成、上传、识别和历史记录功能；后端通过 FastAPI 暴露统一接

口；Agent 负责类型判断、模型路由和推理日志记录；算法层分别实现文字验证码 CRNN+CTC 识别和点选验证码 YOLO+ArcFace 匹配。

课程知识方面，项目覆盖了监督学习、卷积神经网络、循环神经网络、CTC 序列建模、目标检测、度量学习、贪心搜索、置信度决策和可解释 AI 等基础内容。相比只训练一个模型，本项目更强调如何把多个 AI 模块组织为可运行、可展示、可调试的系统。

后续可以从三个方向继续完善：第一，补充更系统的实验数据和可视化图表，形成量化评估；第二，引入更强的 OCR 或 Transformer 视觉模型，提高复杂验证码的泛化能力；第三，完善异常样本处理和安全边界说明，使系统更符合真实 AI 应用的可靠性要求。

指标	数值	备注
推理延迟	300-500ms	CPU 环境 (1 核 2G 可运行)
YOLO 检测准确率	>95%	文字区域召回率
分流准确率	>97%	三策略融合后
ArcFace 匹配准确率	>96%	字符配对正确率
综合准确率	96%	端到端全管线
YOLO 大小	6MB	yolov8n.pt
ArcFace 大小	42MB	arcface_backbone.onnx